

- Senior Manager: Enterprise Cloud — Interview Questionnaire (Part 1)
 - Section 1: Cloud Strategy & Roadmap (5 Questions)
 - Q1. As a Senior Manager, how would you define and execute an enterprise cloud strategy and roadmap for an organization scaling its B2B digital platforms?
 - Q2. How do you design and govern a multi-account AWS setup for an enterprise with multiple business units?
 - Q3. How do you build a cloud adoption framework for teams that are new to cloud? How do you handle resistance?
 - Q4. How do you approach build-vs-buy decisions for cloud tooling at the enterprise level?
 - Q5. How would you lead a cloud-native modernization initiative for an enterprise with a large legacy footprint?
 - Section 2: Cloud Platform Engineering (5 Questions)
 - Q6. How do you implement Terraform at enterprise scale across multiple teams and environments?
 - Q7. How would you design an enterprise EKS platform that serves multiple product teams?
 - Q8. How do you design enterprise CI/CD pipelines that scale across 20+ engineering teams?
 - Q9. How do you govern container platforms at enterprise scale?
 - Q10. How do you ensure platform reliability — what's your approach to SLAs, SLOs, and SLIs?

Senior Manager: Enterprise Cloud — Interview Questionnaire (Part 1)

Role: Senior Manager, Enterprise Cloud | **Location:** Bangalore | **Exp:** 18+

Candidate: Pushparaj Naik — 22+ years | AWS SA + GCP PCA certified

Scope: AWS-focused (Azure excluded per request)

Section 1: Cloud Strategy & Roadmap (5 Questions)

Q1. As a Senior Manager, how would you define and execute an enterprise cloud strategy and roadmap for an organization scaling its B2B digital platforms?

Answer:

I approach cloud strategy through **four pillars**: Business Alignment, Technical Foundation, People, and Governance.

Phase 1 — Discovery & Assessment (Month 1-2)

- Interview business stakeholders to understand revenue goals, growth projections, and pain points
- Assess current infrastructure maturity: what's on-prem, what's in cloud, what's hybrid
- Catalog all workloads with a migration readiness assessment (6 R's: Rehost, Replatform, Refactor, Repurchase, Retain, Retire)
- Identify quick wins vs. strategic bets

Phase 2 — Vision & Roadmap (Month 2-3)

- Define the **target state architecture**: cloud-native, container-first, serverless where appropriate
- Build a 12-month rolling roadmap with quarterly milestones:

Quarter	Focus	Deliverable
Q1	Foundation	Multi-account landing zone, IaC baseline, CI/CD platform
Q2	Migration Wave 1	Rehost/replatform top 5 workloads, establish CloudOps
Q3	Modernization	Containerize key services to EKS, implement observability
Q4	Optimization	Cost governance, security hardening, FinOps practice

Phase 3 — Execution with Governance

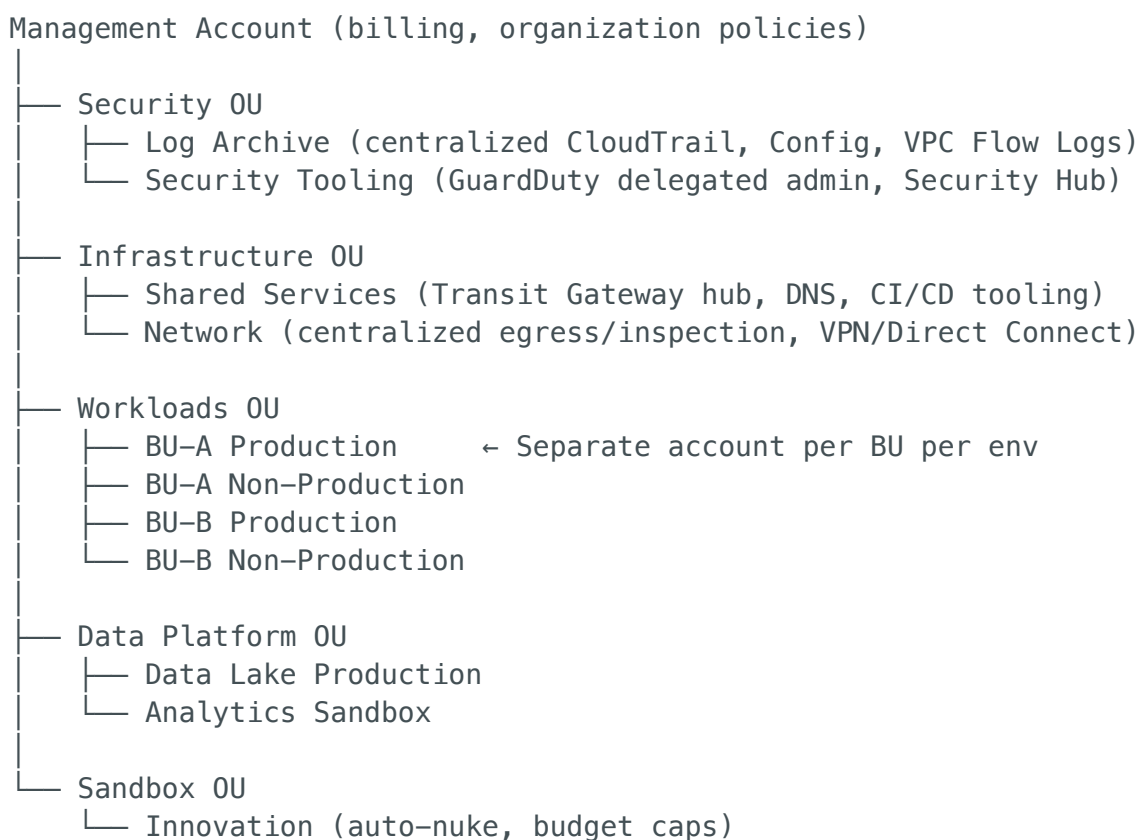
- Establish a Cloud Center of Excellence (CCoE) with representatives from engineering, security, and finance
- Monthly roadmap reviews with executive sponsors — adjust priorities based on business needs
- Track KPIs: migration velocity, cloud spend efficiency, incident reduction, deployment frequency

My experience: At ITC Infotech, I defined the DevOps and IaC blueprint for Rio Tinto's AWS platform — repo structure, branching model, remote state layout, environments, and deployment flow from Dev to Prod. This is exactly the foundational work a cloud strategy requires, scaled to the enterprise level.

Q2. How do you design and govern a multi-account AWS setup for an enterprise with multiple business units?

Answer:

I use **AWS Organizations with Control Tower** as the foundation:



Governance Controls:

Layer	Mechanism	Example
Preventive	SCPs (Service Control Policies)	Block non-approved regions, deny public S3
Detective	AWS Config Rules (org-wide)	Detect unencrypted resources, missing tags
Corrective	Config Auto-Remediation	Auto-enable encryption, auto-tag resources
Financial	AWS Budgets per account	Alert at 80%, hard cap at 100% for sandbox

Key decisions I enforce:

- **Account vending machine:** Terraform module that creates new accounts with baseline security (CloudTrail, Config, GuardDuty) automatically
- **Network topology:** Hub-and-spoke via Transit Gateway — workload accounts don't peer directly
- **IAM Identity Center (SSO):** Federated access — no IAM users, no long-lived access keys
- **Tagging strategy:** Mandatory tags enforced via SCP — **CostCenter**, **Owner**, **Environment**, **DataClassification**

At Rio Tinto, I implemented state isolation per environment with cross-account IAM roles via OIDC — this scales naturally to multi-account enterprise setups.

Q3. How do you build a cloud adoption framework for teams that are new to cloud? How do you handle resistance?

Answer:

Framework — The Three Tracks:

Track 1: Foundation (Platform Team provides)

- Golden path templates: pre-built Terraform modules for VPC, EKS, RDS, S3 with security baked in
- Self-service catalog: teams can deploy approved architectures without tickets
- Reference architectures: documented patterns for web apps, APIs, data pipelines, event-driven systems

Track 2: Enablement (Cloud CoE provides)

- Hands-on workshops: 2-day "Cloud for Developers" bootcamp with real AWS sandbox
- Pairing program: cloud engineers pair with application teams for first 2 sprints
- Internal documentation: confluence space with runbooks, FAQs, architecture decision records
- Office hours: weekly open session for any cloud questions

Track 3: Governance (Automated guardrails)

- Automated security scanning in CI/CD — teams don't need to be security experts
- Cost dashboards per team — visibility drives accountability
- Architecture review for any new service — lightweight, not bureaucratic

Handling Resistance:

- **"Cloud is less secure than on-prem"** → Show AWS compliance certifications (SOC 2, PCI, ISO 27001). Demonstrate that automated scanning catches more issues than manual on-prem audits.
 - **"We'll lose control"** → Multi-account isolation gives MORE control than shared on-prem. Each team gets their own account with guardrails.
 - **"Our team doesn't have skills"** → That's what Track 2 solves. Budget for AWS certifications, pair programming, and gradual ownership transfer.
 - **"Migration is risky"** → Start with non-critical workloads. Build confidence through quick wins before tackling core systems.
-

Q4. How do you approach build-vs-buy decisions for cloud tooling at the enterprise level?

Answer:

I use a **weighted decision matrix** evaluated against five criteria:

Criteria	Weight	Build (Custom)	Buy (Managed/SaaS)
Time to value	25%	Months to build, ongoing maintenance	Days to weeks, vendor maintains
Total cost (3yr)	25%	Dev cost + maintenance + opportunity cost	License + integration cost
Strategic differentiation	20%	Only if it's core IP/competitive advantage	Commodity tooling — don't reinvent
Team capability	15%	Do we have and want to retain this expertise?	Vendor provides expertise
Vendor lock-in risk	15%	Full control, portable	Dependency on vendor roadmap

My general framework:

- **Build:** Core business logic, proprietary data pipelines, competitive differentiators
- **Buy/Use managed:** CI/CD (GitHub Actions), monitoring (Datadog/CloudWatch), secrets (Secrets Manager), container orchestration (EKS not self-managed K8s)

Example from my experience: At Rio Tinto, we chose GitHub Actions over building a custom CI/CD platform. The ROI was clear: OIDC integration with AWS removed credential management overhead, and the team could focus on business logic instead of pipeline infrastructure. But we built custom Terraform modules in-house because IaC patterns were specific to our architecture and needed to encode our governance policies.

Key principle: If it's not your core business, don't build it. A cloud platform team should build the **abstractions and guardrails**, not the underlying tools.

Q5. How would you lead a cloud-native modernization initiative for an enterprise with a large legacy footprint?

Answer:

Strategy: Incremental modernization, not Big Bang

I categorize applications into four buckets and apply different strategies:

Bucket	Criteria	Strategy	Timeline
Lift & Shift	Stable, low change rate, meets SLAs	Rehost to EC2/ECS, minimal changes	2-4 weeks per app
Replatform	Needs scaling, uses managed DB equivalent	Move to RDS, S3, ElastiCache — keep app code	4-8 weeks per app
Refactor	High business value, needs agility	Decompose to microservices on EKS	3-6 months per app
Retire/Replace	Redundant or better SaaS exists	Decommission or adopt SaaS	Varies

Modernization Patterns I've Used:

1. **Strangler Fig** (from my Advantest project): Route traffic through API Gateway, gradually extract microservices from the monolith while it continues running
2. **Database Modernization:** Oracle → Aurora PostgreSQL using DMS with CDC for zero-downtime migration
3. **Containerization:** At Advantest, I led containerization of GLP and FNO using ECS, and designed multi-AZ RDS with DMS-based migration
4. **Event-Driven Architecture:** Replace synchronous point-to-point integrations with EventBridge/SQS for loose coupling

Success Metrics:

- Deployment frequency: monthly → daily
- Lead time for changes: weeks → hours
- MTTR: hours → minutes
- Change failure rate: >15% → <5%

Section 2: Cloud Platform Engineering (5 Questions)

Q6. How do you implement Terraform at enterprise scale across multiple teams and environments?

Answer:

Architecture — Three-Layer Model:

```
Layer 1: Platform Modules (owned by Cloud Platform Team)
├─ Opinionated, security-hardened building blocks
│   ├── networking (VPC, subnets, TGW, endpoints)
│   ├── compute (EKS, ECS, Lambda patterns)
│   ├── database (RDS, DynamoDB, ElastiCache)
│   ├── storage (S3 with encryption, lifecycle, ACL)
│   ├── security (KMS, WAF, GuardDuty, Config)
│   └─ observability (CloudWatch, alarms, dashboards)
Layer 2: Service Compositions (owned by Service Teams)
├─ Compose platform modules for their application
│   ├── payment-service/ (uses: networking + compute + database)
│   └─ analytics-pipeline/ (uses: storage + compute + database)
Layer 3: Environment Configs (owned by DevOps/SRE)
├─ Environment-specific variables
│   ├── dev.tfvars (smaller instances, shorter retention)
│   ├── staging.tfvars (prod-like config)
│   └─ prod.tfvars (full HA, full retention)
```

Governance at Scale:

Concern	Solution
Module versioning	Terraform private registry with semantic versioning
State management	S3 + DynamoDB per account per env, KMS encrypted
Code quality	<code>terraform fmt</code> , <code>terraform validate</code> , <code>tflint</code> in CI
Security scanning	Checkov/tfsec in every PR — block merge on CRITICAL
Drift detection	Scheduled <code>terraform plan</code> every 4 hours, alert on drift
Cost estimation	Infracost in PR comments — developers see cost impact
Documentation	<code>terraform-docs</code> auto-generates module docs

At Rio Tinto, I built this exact pattern — modular Terraform stack for S3, RDS, Glue, SNS, IAM, KMS as reusable patterns. Teams adopted them while maintaining production readiness across all environments.

Q7. How would you design an enterprise EKS platform that serves multiple product teams?

Answer:

Multi-Tenant EKS Architecture:

```
Shared EKS Cluster (production)
├─ Namespace: team-payments      (ResourceQuota: 16 vCPU, 32Gi)
├─ Namespace: team-catalog       (ResourceQuota: 8 vCPU, 16Gi)
├─ Namespace: team-channels      (ResourceQuota: 8 vCPU, 16Gi)
├─ Namespace: platform-ingress  (ALB Ingress Controller)
├─ Namespace: platform-mesh     (Istio control plane)
└─ Namespace: platform-monitoring (Prometheus, Grafana)
```

Platform Team Responsibilities:

- EKS cluster lifecycle (upgrades, patching, scaling)
- Shared add-ons: ingress controller, service mesh, cert-manager, external-secrets
- RBAC templates: teams get admin in their namespace, read-only cluster-wide
- IRSA configuration: each team's pods get scoped IAM roles
- Network policies: default-deny between namespaces, explicit allow-list
- PodDisruptionBudgets and Pod Security Standards enforcement

Product Team Self-Service:

- Teams own their namespace: deploy via CI/CD (Helm charts or Kustomize)
- Standardized Helm chart templates provided by platform team
- GitOps with ArgoCD: teams push to Git, ArgoCD syncs to cluster

Node Architecture:

```
# Managed node groups with Graviton for cost optimization
resource "aws_eks_node_group" "general" {
  instance_types = ["m7g.xlarge"] # Graviton – 20% cheaper
  capacity_type  = "ON_DEMAND"    # Production stability
```

```

scaling_config {
  min_size = 3
  max_size = 20
  desired_size = 6
}
}

# Spot instances for non-critical workloads
resource "aws_eks_node_group" "batch" {
  instance_types = ["m7g.xlarge", "m6g.xlarge", "c7g.xlarge"]
  capacity_type = "SPOT"

  taint {
    key    = "workload-type"
    value  = "batch"
    effect = "NO_SCHEDULE"
  }
}

```

At Advantest, I led containerization using ECS. For enterprise scale with multiple teams, EKS with namespace isolation and GitOps provides the necessary multi-tenancy and self-service capabilities.

Q8. How do you design enterprise CI/CD pipelines that scale across 20+ engineering teams?

Answer:

Platform Approach — CI/CD as a Product:

Rather than each team building their own pipeline, the platform team provides **reusable pipeline templates**:

GitHub Actions — Shared Workflows:

```

# .github/workflows/deploy.yml (in each team's repo)
name: Deploy
on: push
jobs:
  deploy:
    uses: org/platform-workflows/.github/workflows/standard-deploy.yml@v2
    with:
      service-name: payment-service

```

```
environment: production
terraform-version: "1.7.0"
secrets: inherit
```

Standard Pipeline Stages:

1. Build & Test → Compile, unit tests, code coverage
2. Security Scan → SAST (SonarQube), dependency scan (Snyk), container scan (Trivy)
3. Build Artifact → Docker image → ECR (immutable tags, signed with cosign)
4. Deploy to Dev → Automatic on merge to main
5. Integration Test → Automated API/contract tests
6. Deploy to Staging → Automatic after tests pass
7. Performance Test → Load testing (k6/Locust) against staging
8. Deploy to Prod → Manual approval gate → canary/blue-green
9. Smoke Test → Synthetic monitoring post-deploy
10. Notify → Slack/Teams notification with deploy summary

Guardrails for All Pipelines:

- **OIDC authentication** — no stored AWS credentials (implemented at Rio Tinto)
- **Mandatory security scanning** — PRs blocked if CRITICAL vulnerabilities found
- **Infracost** — cost estimation in every Terraform PR
- **Mandatory approvals** — 2 reviewers for production changes
- **Audit trail** — every deployment logged with who, what, when, approval chain

Metrics I Track:

Metric	Target	How Measured
Deployment frequency	Daily per team	GitHub deployments API
Lead time (commit → prod)	< 4 hours	Deployment event timestamps
Change failure rate	< 5%	Rollback count / deploy count
MTTR	< 30 minutes	Incident ticket resolution time

Q9. How do you govern container platforms at enterprise scale?

Answer:

Container Governance Framework:

Domain	Control	Enforcement
Base Images	Only approved, hardened base images	ECR pull-through cache + OPA/Kyverno admission policy
Vulnerability Scanning	Every image scanned before deploy	ECR native scanning + Trivy in CI
Image Signing	Only signed images can run in production	cosign + Kyverno verify-image policy
Resource Limits	Every pod must have CPU/memory limits	OPA policy rejects pods without limits
Security Context	Non-root, read-only filesystem, no privilege escalation	Pod Security Standards (restricted)
Network Isolation	Default-deny between namespaces	Calico NetworkPolicies
Secrets	No Kubernetes Secrets for sensitive data	Secrets Store CSI Driver → Secrets Manager
Registry	Only pull from internal ECR	Kyverno policy blocks external registries

Golden Dockerfile Standards:

```
# APPROVED: Use org-approved base image
FROM 123456789.dkr.ecr.ap-south-1.amazonaws.com/base-images/python:3.11-slim

# Security: non-root user
RUN groupadd -r appuser && useradd -r -g appuser appuser
USER appuser

# Security: no secrets in build
COPY --chown=appuser:appuser requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY --chown=appuser:appuser . .
EXPOSE 8080
CMD ["gunicorn", "app:create_app()"]
```

Q10. How do you ensure platform reliability — what's your approach to SLAs, SLOs, and SLIs?

Answer:

Hierarchy:

- **SLA** (external promise to business): "99.95% availability for customer-facing APIs"
- **SLO** (internal target, tighter than SLA): "99.97% availability, P99 latency < 300ms"
- **SLI** (measured signals): HTTP success rate, request latency, error rate

Implementation:

SLI	Measurement	SLO Target	Alert Threshold
Availability	(200-399 responses) / total responses	99.97%	< 99.95% over 5 min
Latency (P99)	ALB target response time	< 300ms	> 500ms over 5 min
Error Rate	5xx responses / total responses	< 0.1%	> 0.5% over 5 min
Throughput	Requests per second	> baseline $\pm$ 30%	Drop > 50% over 5 min

Error Budget Model:

- Monthly error budget = 100% - SLO = 0.03% = ~13 minutes of downtime allowed
- If budget is healthy → approve risky deployments, run chaos experiments
- If budget is burning → freeze non-critical changes, focus on reliability

At Wipro/HP, I managed production environments with these SRE principles — CloudWatch + Datadog monitoring with threshold-based alerting. At this level, I'd formalize it with error budgets and SLO dashboards visible to engineering leadership.
